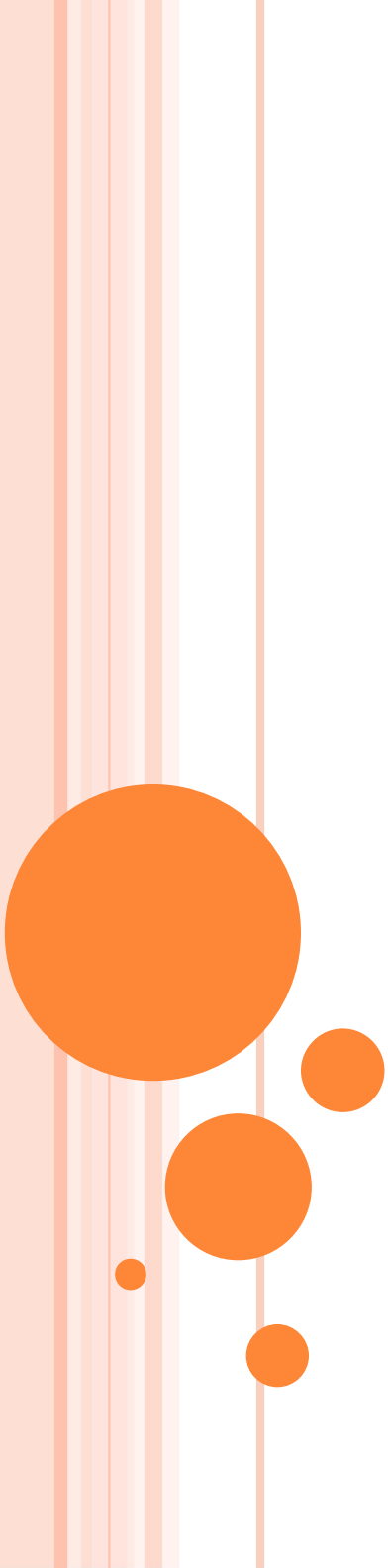




PROGRAMOWANIE GIER

WYKŁAD 1 – WPROWADZENIE

Bogumiła Hnatkowska

KRYTERIA ZALICZENIA

○ Wykład:

- Test zaliczeniowy – pisemny zawierający pytania otwarte, testowe, z luką, sprawdzający wiedzę z zakresu wykładu.
- Terminy – przedostatni wykład (I termin), ostatni wykład (II termin)
- Można dostać dodatkowe punkty za aktywność na wykładach, doliczane do wyniku (max. 10%)
- Ocena ustalana na podstawie liczby otrzymanych punktów:
 < 50% → ndst, < 60% → dst, < 70% → dst+, < 80% → db, < 90% → db+, < 96% → bdb; w przeciwnym przypadku celujący

○ Laboratorium:

- Obecność obowiązkowa (dopuszczalne 2 nieobecności)
- Do przygotowania:
 - Indywidualnie: 4 zadania (z terminem wykonania 2 tygodnie), każde za 10 punktów. Za każdy tydzień spóźnienia ocena jest obniżana o 5 punktów
 - Grupowo (zespoły 2 osobowe): 1 mini-projekt za 20 punktów (Lab 10-14).
- Ocena ustalana na podstawie sumy punktów, zgodnie z formułą:

< 30 → ndst,	[30-35) → dst,	[36-42) → dst+
[42-48) → db	[48-54) → db+	[54-59) → bdb
[59-60] → cel		

○ Ocena końcowa: $0,6 * \text{lab} + 0,4 * \text{wykład}$

○ Obie oceny (lab, wykład) muszą być pozytywne



LITERATURA

- <http://docs.unity3d.com/Manual/index.html>
- [Victor G Brusca](#), Advanced Unity Game Development: Build Professional Games with Unity, C#, and Visual Studio, Apress, 2021
- [Joseph Hocking](#), Unity in Action, Third Edition, Manning Publications, 2022
- Materiały z wykładu



SILNIKI GIER

- Główna część kodu gry komputerowej wraz ze zintegrowanym środowiskiem programistycznym dla twórców gier. Kod może obsługiwać wiele mechanizmów, np. moduł wejścia, moduł graficzny (oświetlenie/cieniowanie), animacje, wykrywanie kolizji etc.
- Przykłady:
 - Unity
 - Unreal Engine
 - GameMaker
 - Godot
 - Cry Engine
 - ...



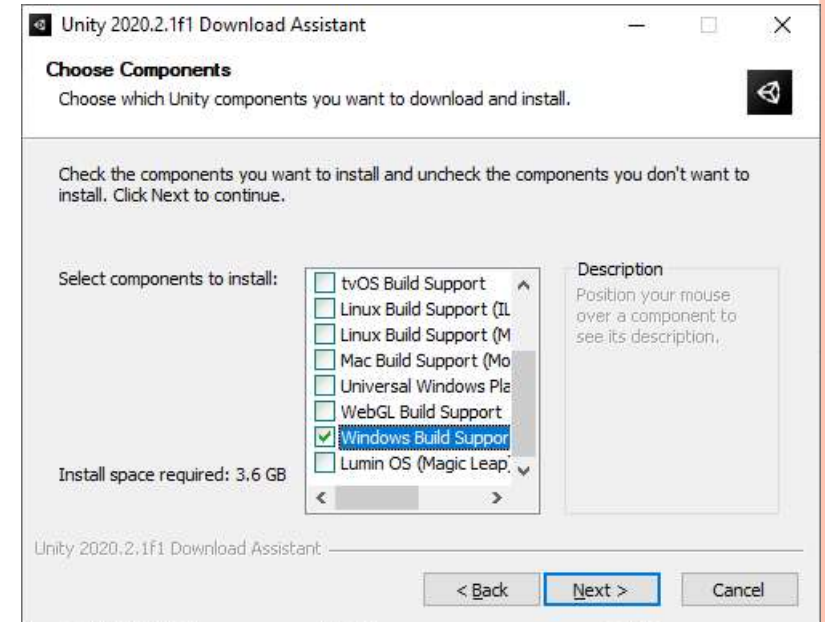
UNITY – WPROWADZENIE

- Wieloplatformowy silnik gier
- Gry 2D, 2.5D, 3D
- Edytor do projektowania gry (wersja darmowa i profesjonalna)
- Skrypty w c# (Visual Studio, Visual Studio Code, JetBrains Rider, ...)
- Możliwość skorzystania z Unity Student Plan



INSTALACJA UNITY

- unity3d.com/download
- Unity Hub – aplikacja, która usprawnia proces zarządzania instalacjami Unity i projektami (wymaga konta Unity Developer Network)
 - Możliwość zarządzania licencjami
 - Przełączanie między wersjami Unity



INNE PRZYDATNE NARZĘDZIA

○ Narzędzia do modelowania 3D:

- **Blender (darmowy)**
- Maya
- 3D Studio Max
- Cinema 4D
- Cheetah3D

○ Narzędzia do grafiki 2D:

- **Gimp (darmowy)**
- Adobe Photoshop
- Corel



PROJEKTY 2D I 3D

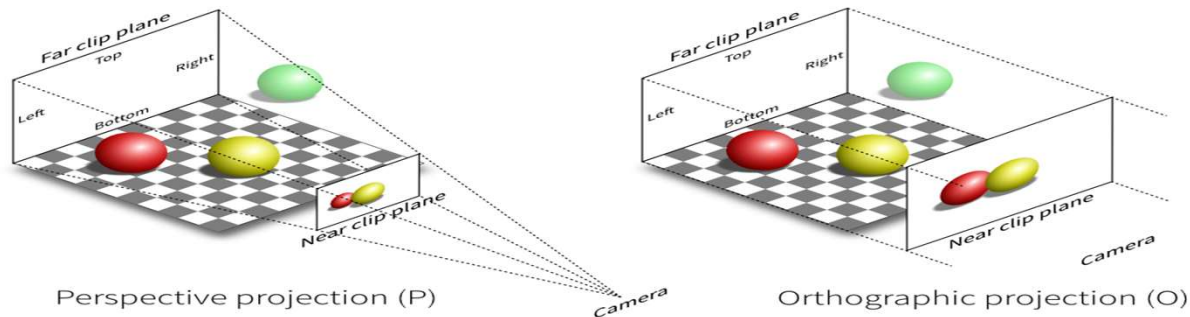
○ Pełne 3D (Full 3D)

- 3 wymiarowa geometria
- Kamera może poruszać się dowolnie po scenie
- Oświetlenie dowolne
- Perspektywa



PROJEKTY 2D I 3D, C.D.

- Ortogonalne 3D (Orthographic 3D)/2.5D:
 - 3 wymiarowa geometria
 - Kamera ortogonalna (a nie perspektywiczna) – obiekty tej samej wielkości bez względu na położenie



PROJEKTY 2D I 3D, C.D.

- Pełne 2D (Full 2D):
 - 2 wymiarowa grafika
 - Kamera bez perspektywy



POTOKI RENDEROWANIA (RENDER PIPELINES)

Potok renderowania wykonuje operacje, które wyświetlają scenę na ekranie. Na wysokim poziomie operacje te obejmują:

- Culling (ukrywanie niewidocznych obiektów)
- Rendering
- Post-processing

Wybór potoku renderowania wpływa na charakterystykę wydajności gry, determinując zestaw platform, na których gra ma być wdrożona.

Dostępne potoki:

- **Wbudowany (Built-in Render Pipeline) – ogólnego przeznaczenia (rekomendowany, działa na wszystkich platformach, w tym VR) z ograniczonymi możliwościami dostosowania**
- Uniwersalny (Universal Render Pipeline, URP) – programowalny, który można łatwo dostosować (optymalizacje)
- High Definition Render Pipeline (HDRP) – programowalny, dla aplikacji wymagających grafiki wysokiej jakości

Porównanie możliwości potoków:

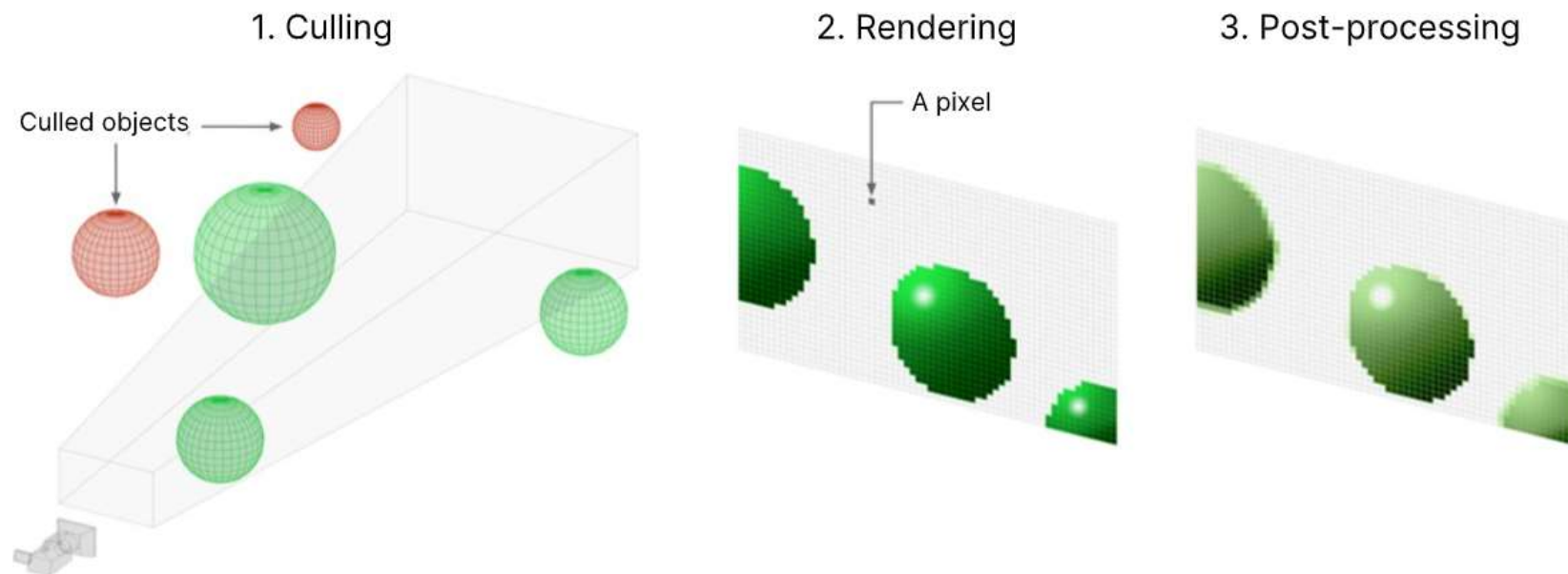
[Unity - Manual: Render pipeline feature comparison \(unity3d.com\)](https://unity3d.com/manual/render-pipeline-feature-comparison)



POTOKI RENDEROWANIA (RENDER PIPELINES)

Potok renderowania wykonuje operacje, które wyświetlają scenę na ekranie. Na wysokim poziomie operacje te obejmują:

- Culling (ukrywanie niewidocznych obiektów)
- Rendering
- Post-processing



Porównanie możliwości potoków:

[Unity - Manual: Render pipeline feature comparison \(unity3d.com\)](http://unity3d.com/manual/Render-pipeline-feature-comparison)



SZABLONY

- 2D, 3D, 2D mobile, 3D mobile – wbudowany potok renderowania
- URP: Universal 2D, Universal 3D, First Person, VR
- HDRP: High Definition 3D
- Inne (przykładowe, np. przykładowe mikro gry)

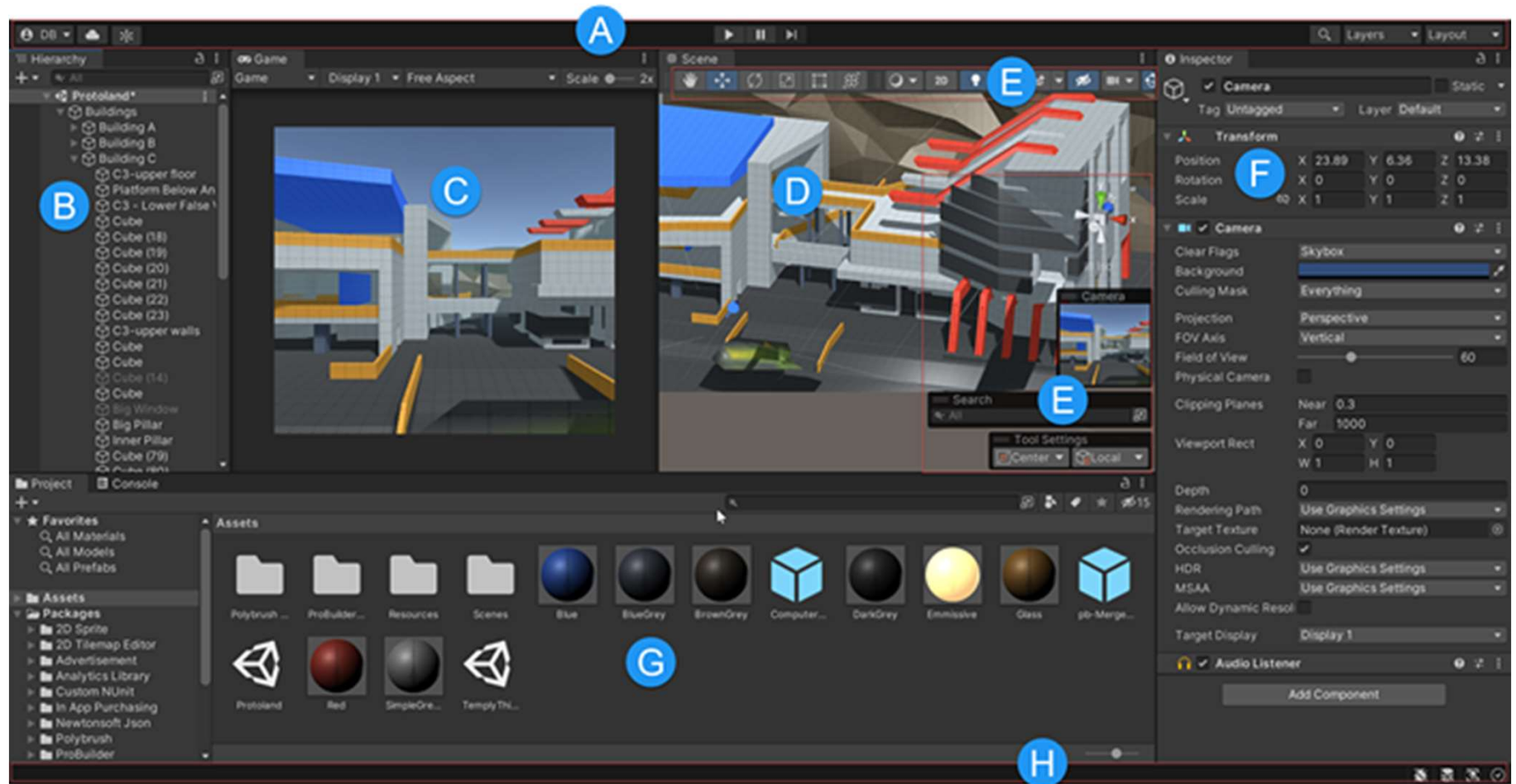
- Do większości zastosowań zaleca się wbudowany potok renderowania (najszerszy zakres platform, możliwość wykorzystania asetów ze sklepu)

[Unity - Manual: Render pipeline feature comparison \(unity3d.com\)](https://unity3d.com/manual/render-pipeline-feature-comparison)

[Unity - Manual: Choosing and configuring a render pipeline and lighting solution \(unity3d.com\)](https://unity3d.com/manual/choosing-and-configuring-a-render-pipeline-and-lighting-solution)



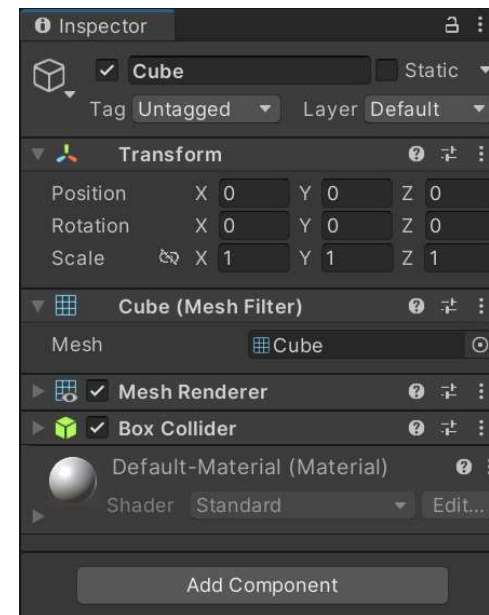
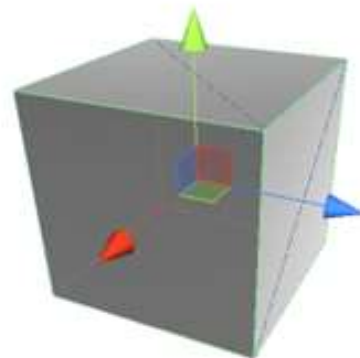
INTERFEJS



POJĘCIA PODSTAWOWE

○ Elementy świata gry:

- Scena (scene) = poziom; definiuje zbiór obiektów gry
- Obiekt gry (GameObject) = kontener komponentów (components):
 - Każdy obiekt gry zawiera komponent Transform (pozycja, rotacja, skala)
 - Może zawierać również inne komponenty (w tym powiązane skrypty), które wpływają na zachowanie gry



POJĘCIA PODSTAWOWE, C.D.

- Metka (Tag) – metainformacja (słowo), która może być powiązana z obiektem gry w celu późniejszej identyfikacji obiektów gry np. w skryptach

`GameObject g = GameObject.FindWithTag("...");`

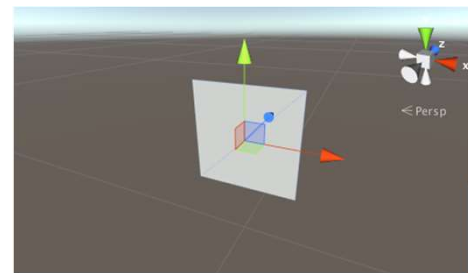
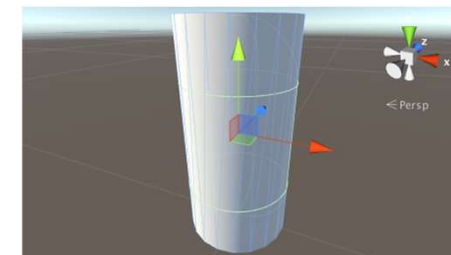
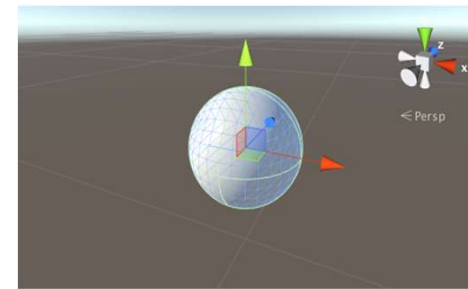
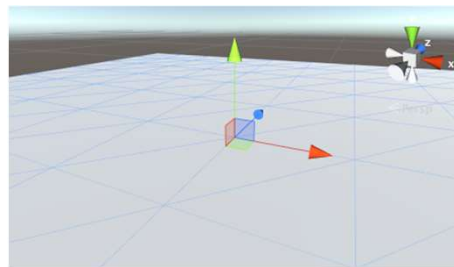
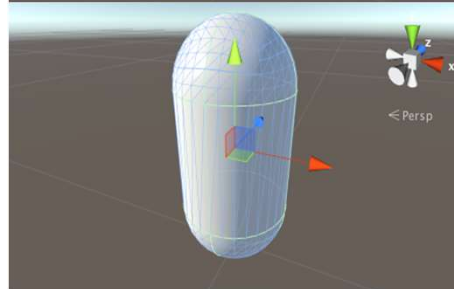
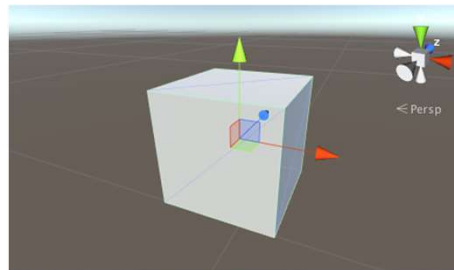
- Poziom (layer) – metainformacja (liczba), która może być wykorzystana np. do optymalizacji wyświetlania czy obsługi kolizji
- Obiekt statyczny (static object) – obiekt, który się nie porusza, a jego wyświetlanie może być zoptymalizowane (ustawienie własności *static*)



POJĘCIA PODSTAWOWE, C.D.

○ Obiekty podstawowe (Primitive objects):

- Kostka (Cube)
- Sfera (Sphere)
- Kapsuła (Capsule)
- Cylinder (Cylinder)
- Płaszczyzna (Plane)
- Ekran (Quad)



PASEK NARZĘDZI (TOOLBAR)



○ Narzędzia transformacji:

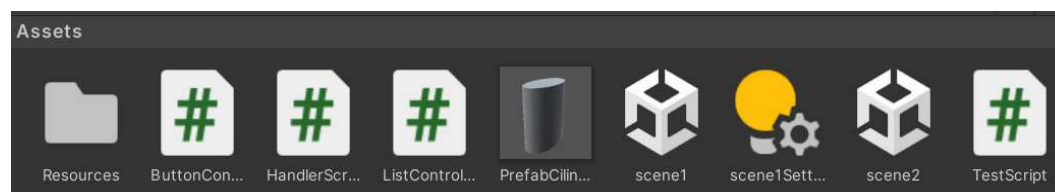
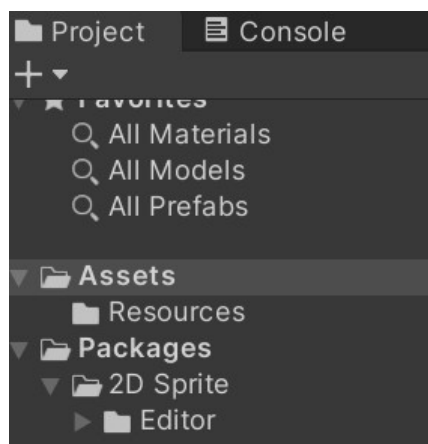


- Hand – przesuwanie sceny
- Move – przesuwanie obiektów
- Rotate – obracanie obiektów
- Scale – skalowanie obiektów
- Rect – skala, przesuwanie
- Transform – złożenie transformacji



POJĘCIA PODSTAWOWE, C.D.

- Zasób (asset) – reprezentacja dowolnego elementu, który może być używany w grze (plik audio, plik wideo, grafika)



- Zasoby są dystrybuowane w postaci pakietów, które można zaimportować do projektu Unity
- Unity jest instalowane z zestawem standardowych pakietów; pakietami można zarządzać (Window/Package Manager)
- Zasoby mogą być pozyskiwane ze sklepu (Asset Store)



POJĘCIA PODSTAWOWE, C.D.

- Podstawowe typy zasobów w grze
 - Obrazy 2D (2D image) – tekstury, tła
 - Modele 3D (3D model) – wirtualne obiekty 3D (oparte o obiekty „siatkowe” (mesh))
 - Materiały (Material) – zestaw informacji (kolor, połysk), który definiuje własności powierzchni obiektu, do którego materiał jest dołączony
 - Animacje – zestaw informacji opisujących ruch powiązanego obiektu
 - System cząstek (Particle system) – mechanizm tworzenia i kontrolowania dużej liczby małych poruszających się obiektów (np. rozpryskująca się woda)
 - Skrypty (scripts) – kody opisujące zachowanie obiektów gry
 - Prefabrykaty (patrz następny slajd)



POJĘCIA PODSTAWOWE, C.D.

- Prefabrykat (Prefab) = szablon opisujący obiekt gry (z zestawem komponentów), na podstawie którego można tworzyć wiele instancji
- Zmiana własności prefabrykatu zmienia je automatycznie we wszystkich instancjach
- Tworzenie prefabrykatów:
 - Asset > Create Prefab – utworzy pusty prefabrykat; przeciągnij obiekt na prefabrykat, aby ustalić jego własności
 - Przeciągnij obiekt do okna assetów (prefabrykat zostanie automatycznie stworzony)



ZACHOWYWANIE PRACY

- Zachowywanie zmian dla sceny (Save)
 - Modyfikacja w hierarchii obiektów
- Zapisywanie zmian specyficznych dla projektu (Save Project)
 - Metki i poziomy zdefiniowane przez użytkownika, siła grawitacji → `TagManager.asset`
 - Ustawienia pracy edytora → `EditorUserSettings.asset`
 - ...



KOMPONENT SKRYPT

- Obiekty gry mogą mieć dowiązane skrypty, które dziedziczą z klasy MonoBehaviour

```
using UnityEngine;  
using System.Collections;
```

Include namespaces for
Unity and Mono classes.

```
public class HelloWorld : MonoBehaviour {
```

The syntax for inheritance

```
    void Start() {  
        // do something once  
    }
```

Put code in here that runs once.

```
    void Update() {  
        // do something every frame  
    }  
}
```

Put code in here that
runs every frame.

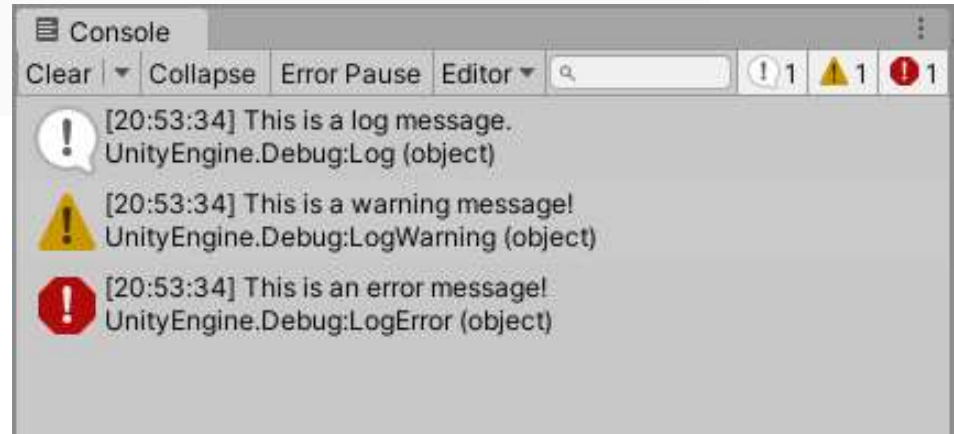
PIERWSZY SKRYPT – HALLO WORLD

- 1) Utwórz pusty obiekt gry: GameObject > CreateEmpty
- 2) Utwórz skrypt: Assets > Create > C# Script
- 3) Przeciągnij skrypt na obiekt gry
- 4) Otwórz skrypt w edytorze i w metodzie Start dopisz: `Debug.Log("Test");`
- 5) Uruchom grę: przycisk Play
- 6) W oknie konsoli sprawdź efekt działania



KLASA DEBUG

```
Debug.Log("To jest wiadomość log.");  
Debug.LogWarning("To jest ostrzeżenie.");  
Debug.LogError("To jest błąd.");  
Debug.Assert(myValue > 0, "myValue musi być większe od 0");  
Debug.DrawLine(Vector3.zero, new Vector3(1, 1, 1), Color.red);  
Debug.DrawRay(Vector3.zero, Vector3.forward * 10, Color.blue);  
Debug.Break();  
Debug.ClearDeveloperConsole();
```



PRZEKIEROWANIE LOGÓW DO PLIKU

```
using UnityEngine;
using System.IO;

public class LogToFile : MonoBehaviour {
    private string logFilePath;

    void OnEnable() {
        logFilePath = Path.Combine(Application.persistentDataPath, "log.txt");
        Application.logMessageReceived += Log;
    }

    void OnDisable() {
        Application.logMessageReceived -= Log;
    }

    void Log(string logString, string stackTrace, LogType type) {
        using (StreamWriter writer = new StreamWriter(logFilePath, true)) {
            writer.WriteLine($"{System.DateTime.Now}: {logString}");
            if (!string.IsNullOrEmpty(stackTrace)) {
                writer.WriteLine(stackTrace);
            }
        }
    }
}
```



SYSTEMY UI W UNITY

- UI Toolkit – najnowszy, oparty na technologii podobnej do CSS i HTML; oferuje zestaw gotowych kontrolek pozwalając je w dużym zakresie stylizować; łatwy do pracy z różnymi rozdzielczościami ekranu; programowalny
- Unity UI (uGUI) – średni; oparty o komponenty z mniej zaawansowanymi funkcjami stylizacji; zestaw typowych kontrolek z możliwością stylizacji; łatwy dla początkujących (używany w przykładach)
- ImGui (Immediate Mode GUI) – najstarszy i najmniej wydajny; wymaga pisania kodu; użyteczny do debugowania (będzie prezentowany później)
- Rekomendacje: [Unity - Manual: Comparison of UI systems in Unity \(unity3d.com\)](https://unity3d.com/manual/comparison-of-ui-systems)



SYSTEM UNITY UI

- HUD (Head-up display) – GUI nałożony na widok gry
- Kanwa (Canvas) – obiekt gry, reprezentujący obszar, w obrębie którego są definiowane elementy UI. Powstaje automatycznie, gdy tworzymy dowolny element UI, jako jego rodzic.
- Elementy UI są reprezentowane jako prostokąty.
- Tryby rysowania kanwy:
 - Screen Space-Overlay – rysuje UI jako grafikę 2D na pierwszym planie; obiekty gry za kanwą
 - Screen Space-Camera – jak wyżej; można zdefiniować jak daleko od kamery jest rysowana kanwa; obiekty gry za lub przed kanwą i mogą ją zasłaniać (w zależności od Z)
 - World Space – umieszcza kanwę na scenie (tak jakby obiekty UI były częścią sceny 3D)



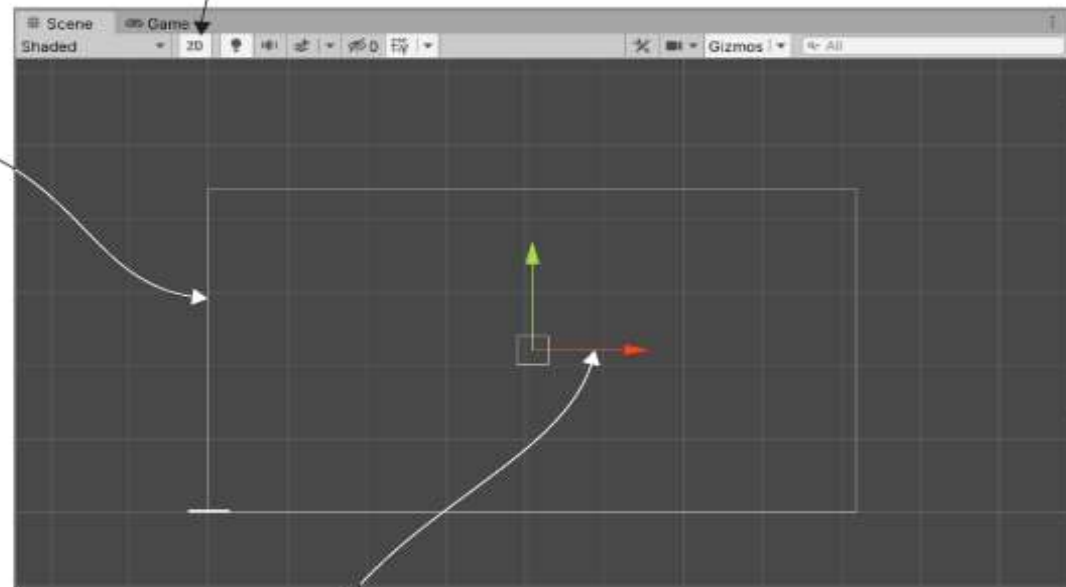
SYSTEM UI

2D view mode: switch to this view when working in 2D (which includes working in the UI).

Canvas object displayed in the Scene view

It's scaled very large because 1 unit in the scene = 1 pixel on the UI.

The borders of the canvas scale to match the game's screen.



If you see the colored arrows of the manipulator, the Rect tool is *not* on. That tool button is in the top-left corner of Unity; you will see blue dots on every corner of a 2D object.



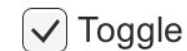
WIZUALNE ELEMENTY UI

- Tekst (Text/Label) – pozwala zdefiniować tekst do wyświetlania, manipulować jego czcionką, stylem czcionki, rozmiarem, wyrównaniem etc.
- Obraz (Image) – pozwala wypełnić obszar kolorem, materiałem, lub grafiką 2D:
 - Simple – wypełnia prostokąt obrazkiem bez zachowania skali
 - Sliced – wypełnia prostokąt obrazkiem rozciągając tylko wnętrze (rogi i krawędzie nienaruszone)
 - Tiled – jeżeli obrazek nie wypełnia prostokąta, powiela go (traktuje jak kafelek)
 - Filled – podobny do simple, ale można sterować kierunkiem i zakresem wypełnienia (np. tylko 50% zaczynając od lewej)
- Raw Image (surowy obraz) – dla obiektów nie wchodzących w interakcje; może być wypełniony dowolną teksturą

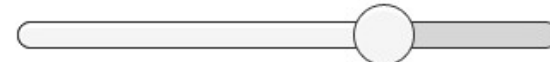


INTERAKTYWNE ELEMENTY UI

- Przycisk (Button) – ma powiązane zdarzenie `OnClick`, które pozwala zdefiniować reakcję na kliknięcie
- Przełącznik (Toggle) – `Is On` pozwala ustalić początkową wartość przełącznika; ma powiązane zdarzenie `OnValueChanged`
- Grupa przełączników (Toggle Group) – grupa przełączników, z których tylko jeden może być włączony
- Suwak (Slider) – `Value` (reprezentuje aktualną wartość), którą można przesuwac między wartościami `Min` i `Max`; ma powiązane zdarzenie `OnValueChanged`



Choose a character



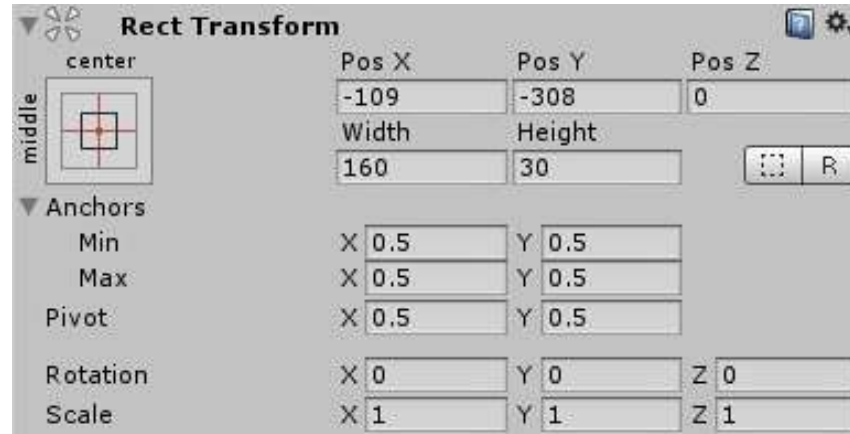
INTERAKTYWNE ELEMENTY UI, C.D.

- Pasek przewijania (Scrollbar) – zwykle używany razem z innymi elementami do ich przewijania (ScrollView)
- Lista rozwijalna (Dropdown) – lista opcji, reprezentowanych tekstem lub obrazkami; ma powiązane zdarzenie OnValueChanged
- Pole tekstowe (Input Field) – edytowalne pole; możliwość zdefiniowania zdarzeń związanych ze zmianą tekstu i zakończeniem wpisywania
- Widok przewijany (ScrollView) – okno z funkcją przewijania do pokazywania długich komunikatów
- Panel (Panel) – „zbiornik” na elementy



POZYCJONOWANIE ELEMENTÓW UI

- Obiekty UI mają komponent Rect Transform, a nie Transform (jak pozostałe).



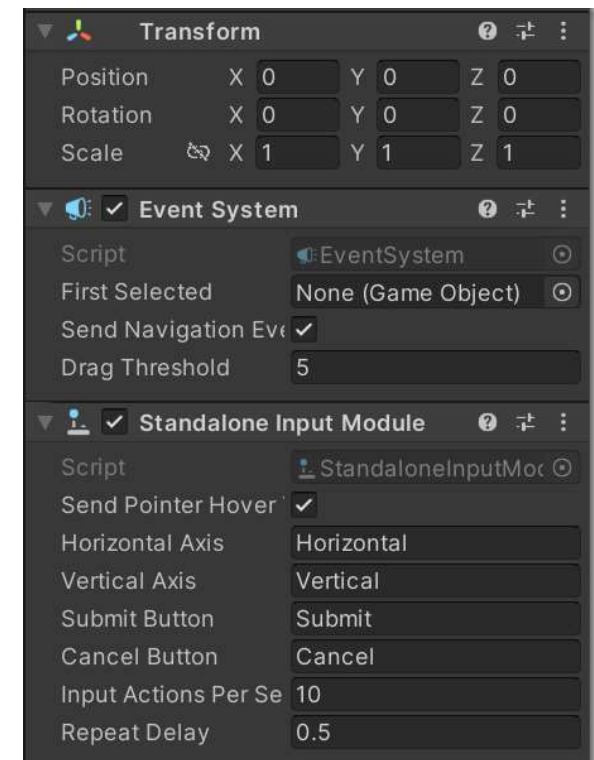
LAYOUT KOMPONENTÓW UI

- Elementy UI mogą mieć dodany komponent zarządzający układem (layoutem).
- Do najczęściej używanych komponentów należą:
 - Horizontal Layer Group
 - Vertical Layer Group
 - Grid Layer Group



SYSTEM ZDARZEŃ (EVENT SYSTEM)

- Komponent odpowiedzialny za mechanizm wysyłania zdarzeń pochodzących od urządzeń wejścia (klawiatura, mysz, ekran dotykowy) do obiektów aplikacji; zaimplementowany w c# w oparciu o interfejsy
- Współpracuje z modułami wejściowymi (Input Modules); w danym momencie może być aktywny tylko jeden moduł
- Predefiniowane moduły wejściowe:
 - Samodzielny (Standalone Input Module) – współpraca przede wszystkim z myszą/klawiaturą
 - Dotykowy (Touch Input Model) – współpraca z ekranem dotykowym



SYSTEM ZDARZEŃ (EVENT SYSTEM), C.D.

Przykładowe interfejsy powiązane ze zdarzeniami dla obiektów UI:

- IPointerEnterHandler - OnPointerEnter – Wywoływany, gdy wskaźnik myszy wejdzie w obszar obiektu
- IPointerExitHandler - OnPointerExit – Wywoływany, gdy wskaźnik opuści obszar obiektu
- IPointerDownHandler - OnPointerDown – Wywoływany, gdy wskaźnik zostanie naciśnięty w obszarze obiektu
- IPointerUpHandler - OnPointerUp – Wywoływany, gdy pointer jest zwolniony
- IPointerClickHandler - OnPointerClick – Wywoływany, gdy wskaźnik jest „kliknięty” (również zwolniony) w obszarze obiektu
- ...



SYSTEM ZDARZEŃ (EVENT SYSTEM), C.D.

- Komponenty UI mogą reagować na zdarzenia zdefiniowane na poprzednim slajdzie
- Sposób reakcji można określić w kodzie lub z wykorzystaniem Inspektora
- Przykład implementacji interfejsu; skrypt dołączony do przycisku (do którego referencja jest również ustalona w oknie Inspektora)

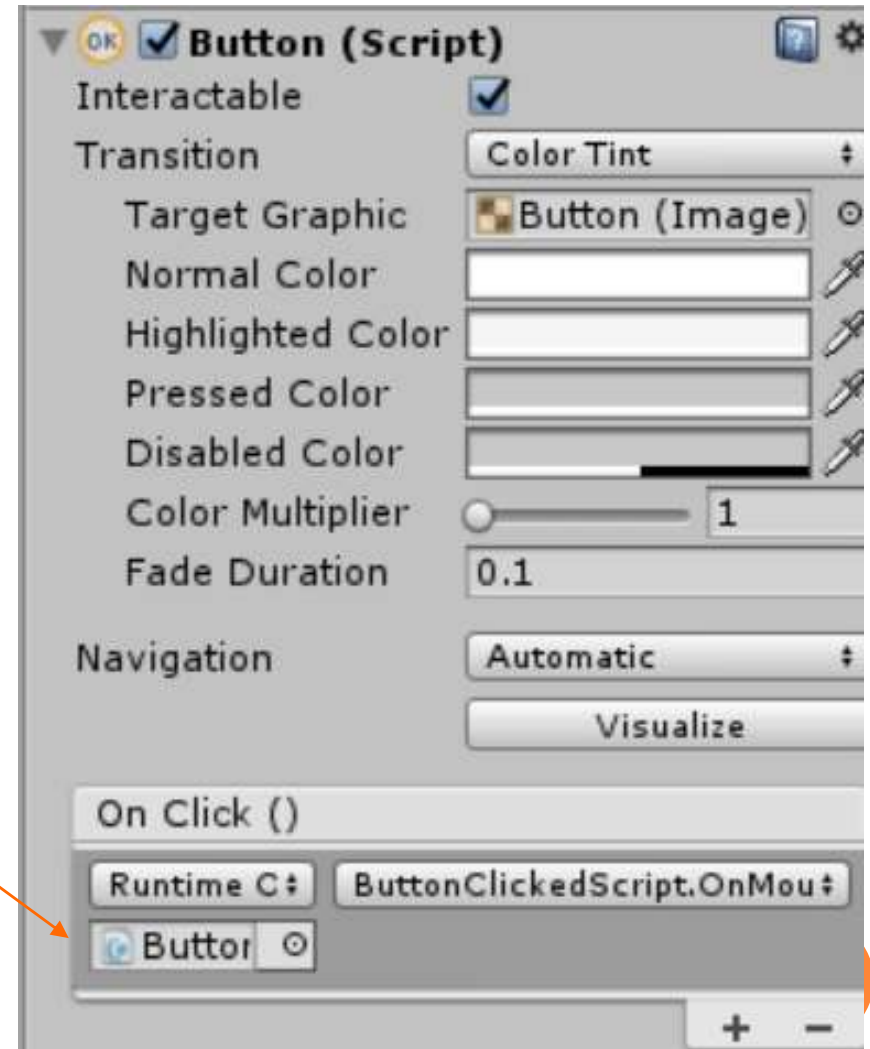
```
using UnityEngine.EventSystems;

public class TestScript : MonoBehaviour, IPointerEnterHandler {
    public Button but;
    public void OnPointerEnter(PointerEventData data){
        Vector3 pos = but.transform.position;
        pos.x += 100;
        but.transform.position = pos;
    }
}
```



SYSTEM ZDARZEŃ (EVENT SYSTEM), C.D.

- Definicja obsługi zdarzenia naciśnięcia przycisku z wykorzystaniem Inspektora
- W oknie referencji ustawiamy obiekt z metodą obsługi zdarzenia `onClick()`

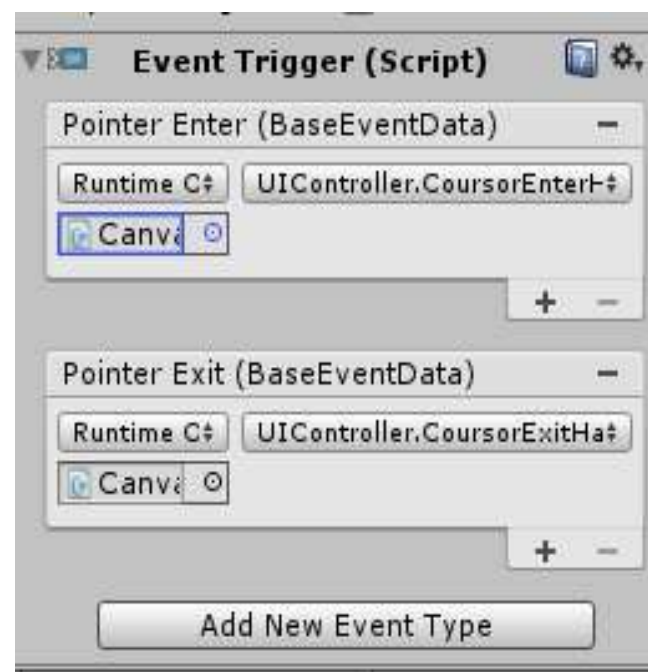


SYSTEM ZDARZEŃ (EVENT SYSTEM), C.D.

- Jeżeli zdarzenia nie widać, należy do komponentu UI dodać komponent Event > EventTrigger oraz go skonfigurować, tzn. określić typ zdarzenia oraz funkcję obsługi
- Przykład – gdy kursor najedzie na tekst, jego kolor zmieni się na czerwony; gdy go opuścimy – na biały.

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections;

public class UIController : MonoBehaviour {
    public Text text;
    public void CursorEnterHandler(){
        text.color = Color.red;
    }
    public void CursorExitHandler(){
        text.color = Color.white;
    }
}
```



SYSTEM ZDARZEŃ (EVENT SYSTEM), C.D.

- Definicja własnych komunikatów (mogą być wysyłane do dowolnych typów obiektów):
 - Przestrzeń UnityEngine.EventSystems ma zdefiniowany interfejs IEventSystemHandler
 - Interfejsy można rozszerzać:

```
public interface TestInterface: IEventSystemHandler {  
    void Message (string txt);  
}
```
 - Każdy byt, który implementuje interfejs, może być adresatem komunikatu

```
public class TestController : MonoBehaviour, TestInterface {  
    public void Message(string txt){ Debug.Log(txt); }  
}
```
 - Komunikat może być wysłany z wykorzystaniem statycznej metody klasy ExecuteEvents

```
ExecuteEvents.Execute<TestInterface> (target, "test",  
    (x, y) => x.Message (y));
```

```
ExecuteEvents.Execute<TestInterface> (target, null,  
    (x, y) => x.Message ("test"));
```

target – obiekt gry, do którego chcemy przesłać komunikat

null – obiekt z parametrami zdarzenia (BasedEventData eventData)



SYSTEM ZDARZEŃ (EVENT SYSTEM), C.D.

- Definicja własnego komunikatu i funkcji obsługi

```
using UnityEngine.EventSystems;

public interface TestInterface: IEventSystemHandler {
    void Message ();
}

public class ImageController : MonoBehaviour, TestInterface {
    public void Message () {
        Debug.Log ("Message received");
        transform.Rotate (new Vector3 (35, 30, 50));
    }
}
```

- Wysyłanie komunikatu do wskazanego obiektu

```
public GameObject cube;

public void CursorEnterHandler(){
    text.color = Color.red;
    // x - handler; y - data
    ExecuteEvents.Execute<TestInterface>(cube, null, (x,y)=>x.Message());
}
```



PROGRAMOWA OBSŁUGA WEJŚCIA (NIE POTRZEBUJE SYSTEMU ZDARZEŃ)

- Wykorzystanie klasy Input do odczytania wybranych akcji użytkownika (naciśnięcie strzałek/"adws", naciśnięcie klawisza Enter) w metodzie Update()

```
float x = Input.GetAxis ("Horizontal");  
float z = Input.GetAxis ("Vertical");  
print ("Horizontal:" + x + " Vertical: " + z);  
  
if (Input.GetButton ("Submit"))  
    print ("submit button");
```

- Fire1 – lewy Ctrl (lewy przycisk myszy)
- Fire2 – lewy Alt (prawy przycisk myszy)



MENADŻER WEJŚCIA INPUT MANAGER

- Edit > Project
settings > Input
Manager

```
void Update () {  
    if (Input.GetButtonDown("Fire1"))  
        print (Input.mousePosition);  
}
```

